

---

# PROCESOS DE INGENIERÍA DEL SOFTWARE 2



UNIVERSIDAD  
DE ALMERÍA

## **PRIS G5 Métricas de Calidad de Proyecto Software**

---

- ISMAEL FERNÁNDEZ MÉNDEZ
- FRANCO SERGIO PEREYRA
- DAVID CASADO FERNÁNDEZ
- DAVID GRANADOS PÉREZ

GRUPO 5

# ÍNDICE

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 1. Introducción a la medición de calidad.....                                    | 3  |
| 2. Modelo de calidad ISO/IEC 25010.....                                          | 4  |
| 3. Clasificación de métricas de software.....                                    | 5  |
| 3.1 Métricas de diseño.....                                                      | 5  |
| 3.2 Métricas de complejidad.....                                                 | 5  |
| 3.3 Métricas de tamaño.....                                                      | 5  |
| 3.4 Importancia de la combinación de métricas.....                               | 6  |
| 4. Métricas de Diseño en Programación Orientada a Objetos: Las CK Metrics.....   | 7  |
| 4.1 WMC (Weighted Methods per Class).....                                        | 7  |
| 4.2 DIT (Depth of Inheritance Tree).....                                         | 9  |
| 4.3 NOC (Number of Children).....                                                | 10 |
| 4.4 CBO (Coupling Between Objects).....                                          | 10 |
| 4.5 RFC (Response For a Class).....                                              | 12 |
| 4.6 LCOM (Lack of Cohesion of Methods).....                                      | 12 |
| 4.7 Interpretación Conjunta y Perfiles de Riesgo de CK Metrics.....              | 14 |
| 5. Métricas de complejidad.....                                                  | 16 |
| 5.1 Complejidad Ciclomática (McCabe).....                                        | 16 |
| 5.2 Carga Cognitiva (Complejidad Percibida).....                                 | 18 |
| 5.3 Relación con la Deuda Técnica.....                                           | 19 |
| 6. Métricas de tamaño.....                                                       | 20 |
| 6.1 Líneas de código (LOC).....                                                  | 20 |
| 6.2 Puntos de Función (Function Points - FP)Definición Detallada.....            | 21 |
| 6.3 Comparación Sintética: LOC vs. Function Points.....                          | 22 |
| 6.4 Interpretación en la Ingeniería de Software Moderna.....                     | 22 |
| 6.5 Relación con el Valor de Negocio y la Calidad.....                           | 23 |
| 7. Uso práctico de métricas en proyectos reales.....                             | 24 |
| 7.1 Integración en el ciclo de desarrollo (CI/CD).....                           | 24 |
| 7.2 Identificación y Gestión de Zonas Críticas (Hotspots).....                   | 24 |
| 7.3 Umbrales (Thresholds) y el Criterio de Calidad del Código.....               | 25 |
| 7.4 Uso Estratégico para Priorización Técnica (Refactorización).....             | 26 |
| 7.5 Cuantificación de la Deuda Técnica.....                                      | 27 |
| 8. Métricas y negocio: impacto económico (ROI).....                              | 28 |
| 8.1 Relación Intrínseca entre Calidad, Complejidad y Coste de Mantenimiento..... | 28 |
| 8.2 La Dominancia del Coste de Mantenimiento.....                                | 29 |
| 8.3 El ROI Positivo de la Inversión en Calidad del Software.....                 | 29 |
| 8.4 Gestión del Riesgo y la Toma de Decisiones Estratégicas.....                 | 30 |
| 8.5 Impacto de la Calidad en la Sostenibilidad y Competitividad del Negocio..... | 30 |
| 8.6 Métricas como Herramienta de Gobernanza y Alineación Estratégica.....        | 31 |
| 9. Gobernanza y Dashboard de Calidad.....                                        | 32 |

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 9.1 Gobernanza de la Calidad del Software: Fundamentos y Alcance.....   | 32 |
| 9.2 Dashboards de Calidad: La Visualización del Estado del Sistema..... | 32 |
| 9.3 Quality Gates: La Defensa Activa contra la Deuda Técnica.....       | 33 |
| 9.4 Métricas como Herramienta de Control Continuo.....                  | 34 |
| 9.5 Relación Crítica con Sistemas Reales y Legado.....                  | 34 |
| 10. Bibliografía.....                                                   | 35 |

## 1. Introducción a la medición de calidad

En ingeniería del software, la medición constituye el mecanismo fundamental que permite transformar el desarrollo de software en un proceso controlado, repetible y gestionable. A diferencia de otras disciplinas de ingeniería, donde las propiedades físicas pueden observarse directamente, el software es un producto intangible cuya calidad debe inferirse a partir de indicadores cuantitativos.

Las métricas de software surgen como respuesta a esta necesidad, proporcionando una forma objetiva de evaluar atributos como la complejidad, la mantenibilidad o el acoplamiento. Sin embargo, medir no es un fin en sí mismo, sino un medio para apoyar la toma de decisiones técnicas.

Fenton y Bieman señalan que el principal objetivo de la medición es reducir la incertidumbre en el proceso de desarrollo, permitiendo tomar decisiones basadas en evidencia en lugar de intuición. No obstante, también advierten que una métrica mal interpretada puede conducir a conclusiones erróneas y, en consecuencia, a decisiones perjudiciales para el sistema.

En la práctica, la ausencia de métricas genera problemas similares a los observados en el código heredado: falta de visibilidad, dificultad para evaluar impactos y aumento del riesgo al introducir cambios. Sin datos cuantitativos, el sistema se convierte en una “caja negra” donde cualquier modificación implica incertidumbre.

Desde una perspectiva profesional, las métricas permiten:

- **Evaluar la calidad del diseño** antes de que aparezcan defectos en producción.
- **Detectar deuda técnica** y priorizar refactorizaciones.
- **Estimar costes de mantenimiento**, que suelen representar la mayor parte del ciclo de vida del software.
- **Reducir riesgos**, especialmente en sistemas complejos o críticos.

Sin embargo, es importante destacar que las métricas no deben utilizarse de forma aislada. Una métrica por sí sola no proporciona una visión completa del sistema, ya que la calidad del software es un concepto multidimensional.

Por tanto, el valor real de las métricas no reside en el número obtenido, sino en su interpretación dentro de un contexto técnico y organizativo. Esta idea es clave para evitar un uso incorrecto, como evaluar la productividad de los desarrolladores únicamente mediante valores cuantitativos, lo cual ignora aspectos fundamentales como la calidad del diseño o la complejidad del problema.

En resumen, la medición en software no busca únicamente cuantificar, sino **facilitar la comprensión del sistema y apoyar decisiones que mejoren su calidad y sostenibilidad a largo plazo**.

## 2. Modelo de calidad ISO/IEC 25010

La norma ISO/IEC 25010 define un modelo de calidad del software ampliamente utilizado en la industria para evaluar productos software de manera estructurada. Este modelo establece un conjunto de características y subcaracterísticas que permiten analizar la calidad desde diferentes perspectivas.

Entre las características principales se encuentran:

- **Mantenibilidad**
- **Fiabilidad**
- **Eficiencia**
- **Seguridad**
- **Usabilidad**
- **Compatibilidad**
- **Portabilidad**

Cada una de estas características se descompone en sub características medibles. Por ejemplo, la mantenibilidad incluye:

- Analizabilidad
- Testabilidad
- Modificabilidad
- Reusabilidad
- Modularidad

Estas subcaracterísticas son especialmente relevantes en el contexto de métricas de software, ya que permiten traducir conceptos abstractos de calidad en propiedades observables y cuantificables.

Tal como se observa en sistemas de código heredado, la falta de testabilidad y analizabilidad dificulta la evolución del software y aumenta el coste de mantenimiento . Esto demuestra que la calidad no es un concepto teórico, sino un factor directamente relacionado con la viabilidad técnica y económica del sistema.

Las métricas de software actúan como un puente entre este modelo conceptual y la realidad del código. Por ejemplo:

- **Alta complejidad ciclomática** → reduce la testabilidad y la analizabilidad.
- **Alto acoplamiento (CBO)** → reduce la modularidad y la reusabilidad.
- **Baja cohesión (LCOM alto)** → dificulta la mantenibilidad.

De este modo, las métricas permiten evaluar indirectamente las características de calidad definidas por la norma ISO/IEC 25010.

Desde una perspectiva de ingeniería, esto implica que:

- No se mide el código para medirlo.
- Se mide para evaluar atributos de calidad definidos por un estándar.
- Se toman decisiones basadas en cómo las métricas afectan a dichas características.

Además, el modelo ISO/IEC 25010 permite conectar la calidad técnica con el impacto en el negocio. Por ejemplo:

- Baja mantenibilidad → mayor coste de evolución.
- Baja fiabilidad → mayor número de fallos en producción.
- Baja seguridad → riesgo legal y reputacional.

Esto es especialmente relevante en entornos empresariales, donde la calidad del software no solo afecta al producto, sino también a la sostenibilidad del negocio.

En conclusión, el modelo ISO/IEC 25010 proporciona el marco teórico necesario para interpretar correctamente las métricas de software, permitiendo relacionar valores numéricos con atributos reales de calidad y, en última instancia, con decisiones técnicas y estratégicas.

### 3. Clasificación de métricas de software

Las métricas de software pueden clasificarse en función del aspecto del sistema que analizan. Esta clasificación permite abordar la calidad desde múltiples dimensiones, evitando una visión parcial del sistema.

#### 3.1 Métricas de diseño

Las métricas de diseño evalúan la estructura interna del software, especialmente en sistemas orientados a objetos. Su objetivo es analizar cómo están organizadas las clases, cómo interactúan entre sí y cómo se distribuyen las responsabilidades.

Estas métricas permiten detectar problemas estructurales como:

- Alto acoplamiento.
- Baja cohesión.
- Uso incorrecto de la herencia.

Las métricas CK (Chidamber y Kemerer) constituyen el conjunto más relevante en esta categoría.

#### 3.2 Métricas de complejidad

Las métricas de complejidad miden la dificultad lógica del código. Su objetivo es estimar:

- La dificultad de comprensión.
- El esfuerzo necesario para testear.
- La probabilidad de errores.

La complejidad ciclomática es la más utilizada, ya que cuantifica el número de caminos independientes de ejecución.

#### 3.3 Métricas de tamaño

Las métricas de tamaño miden el volumen del software desarrollado.

Las más conocidas son:

- Líneas de código (LOC)
- Puntos de función

Aunque son útiles para estimación, presentan limitaciones importantes, ya que no reflejan directamente la calidad del software.

### 3.4 Importancia de la combinación de métricas

Ninguna métrica por sí sola es suficiente para evaluar la calidad de un sistema.

Por ejemplo:

- baja complejidad + alto acoplamiento → sistema difícil de modificar
- baja cohesión + tamaño reducido → diseño incorrecto

Por ello, el análisis debe realizarse combinando distintas métricas, interpretándose dentro del contexto del sistema.

## 4. Métricas de Diseño en Programación Orientada a Objetos: Las CK Metrics

El aseguramiento de la calidad del software comienza en la fase de diseño. En el paradigma de la Programación Orientada a Objetos (POO), la estructura y las relaciones entre clases son cruciales para la mantenibilidad, extensibilidad y robustez del sistema. Las **CK Metrics** (Chidamber y Kemerer, 1994) constituyen un conjunto fundamental y ampliamente adoptado para evaluar la calidad intrínseca del diseño en sistemas basados en clases.

Estas métricas ofrecen una visión cuantitativa de propiedades esenciales como la **complejidad**, el **acoplamiento**, la **cohesión** y el uso efectivo de la **herencia**. Su objetivo primario es identificar *smells* (olores) de diseño y potenciales puntos débiles antes de que estos se traduzcan en altos costes de mantenimiento, depuración o evolución.

### 4.1 WMC (Weighted Methods per Class)

#### Definición Detallada:

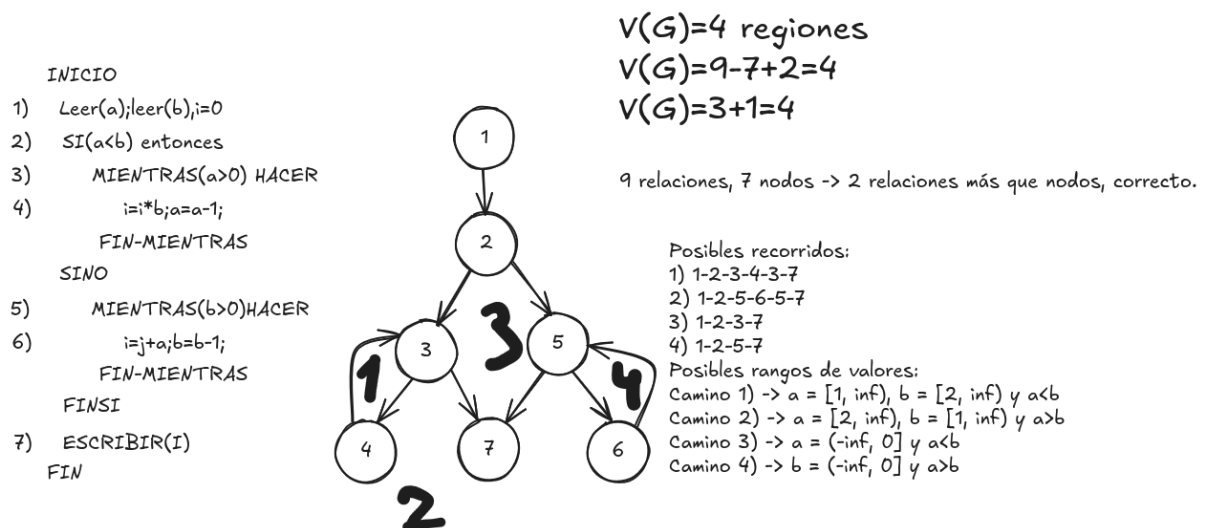
El WMC se define como la suma de las complejidades de todos los métodos definidos dentro de una clase. Aunque teóricamente se puede usar cualquier medida de complejidad, la práctica estándar y más común es utilizar la **Complejidad Ciclomática (CC)** de McCabe para cada método.

Recuérdese que la complejidad ciclomática se calcula como:

$$V(G) = E - N + 2 = P + 1 = R$$

Donde E es el número de aristas, N el número de Nodos, P el número de nodos predicho (donde se toman decisiones) y R las regiones o áreas encerradas.

Un ejemplo de complejidad ciclomática es el siguiente:



Si se considera que todos los métodos tienen una complejidad unitaria, entonces WMC es simplemente el número de métodos (NOM) de la clase.

#### Propósito:

Sirve como un indicador de la complejidad y el tamaño global de la clase, reflejando el esfuerzo requerido para desarrollar, comprender y mantener dicha clase.

#### Interpretación y Consecuencias de Diseño:

- **WMC bajo:** Sugiere una clase enfocada, simple y fácil de comprender. Es



consistente con el principio de Alta Cohesión.

- **WMC alto:** Indica que la clase tiene demasiados métodos, o que varios de sus métodos son internamente muy complejos (alta complejidad ciclomática). Esto es un síntoma claro de la **violación del Principio de Responsabilidad Única (SRP)**, donde la clase asume múltiples responsabilidades o tiene un ámbito de acción demasiado amplio.

#### **Impacto Crucial en la Calidad del Software:**

Un WMC excesivamente alto impacta negativamente:

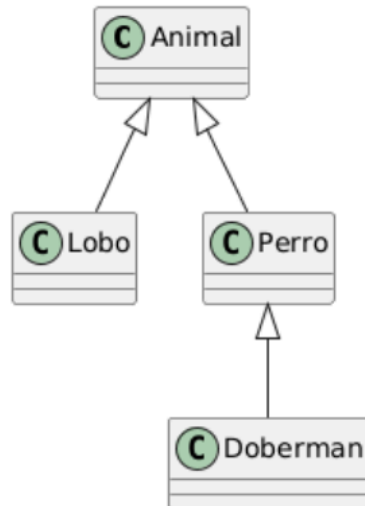
- **Mantenibilidad:** La modificación de la clase se vuelve riesgosa, ya que un cambio podría afectar a un gran número de métodos internos o complejos.
- **Testeabilidad:** Se requiere un número mucho mayor de casos de prueba para cubrir adecuadamente todos los caminos de ejecución, incrementando la dificultad y el coste del testing unitario e integrado.
- **Fiabilidad:** A mayor complejidad, mayor es la probabilidad de introducir y ocultar defectos o errores lógicos.

## 4.2 DIT (Depth of Inheritance Tree)

### Definición Detallada:

Mide la longitud del camino desde la clase actual hasta la raíz de su jerarquía de herencia (generalmente una clase base o **Object**). Si la clase no hereda de ninguna otra (es una clase raíz en el diseño específico, excluyendo Object), su DIT es 0.

### Ejemplo:



DIT(Animal) = 0

DIT(Lobo) = 1

DIT(Perro) = 1

DIT(Doberman) = 2

DIT total = 2

### Propósito:

Evaluar el grado de reutilización y el nivel de influencia que las clases ancestrales ejercen sobre el comportamiento y la implementación de la clase actual.

### Interpretación y Trade-offs:

- **DIT bajo (0 o 1):** Sistema con baja profundidad de herencia. Implica menor reutilización de código mediante herencia, pero el comportamiento de la clase es predecible y local.
- **DIT alto (ejemplo: > 5):** Indica un alto grado de herencia y, potencialmente, una mayor reutilización de funcionalidad. Sin embargo, esto introduce una gran **complejidad cognitiva** y de análisis.

### Problemas Derivados de un DIT Excesivamente Alto:

- **Comportamiento Difícil de Entender:** El comportamiento final de la clase hija es el resultado de la combinación de sus propios métodos más la herencia y posible sobrescritura de métodos de múltiples ancestros.
- **Efectos Colaterales (Side-effects):** La modificación de una clase base muy alta en la jerarquía (un ancestro con DIT bajo) puede tener efectos en cascada e inesperados en un gran número de clases descendientes (con DIT alto), haciendo el sistema frágil.

### Impacto en la Calidad del Software:

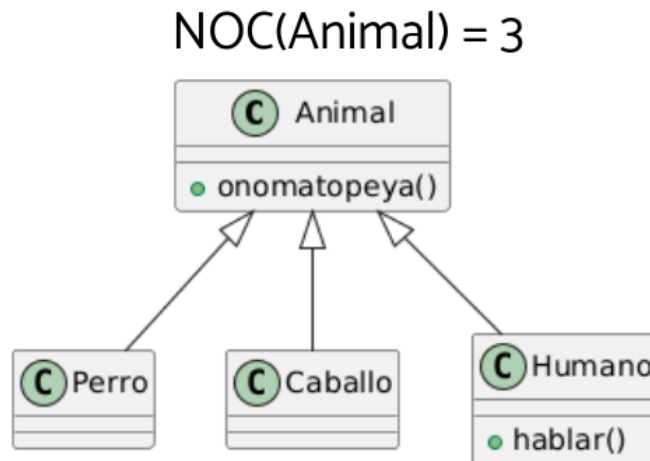
- **Analizabilidad:** La comprensión de la clase se hace modular, requiriendo el análisis de múltiples niveles de la jerarquía.
- **Complejidad Cognitiva:** Aumenta el esfuerzo mental necesario para mapear el comportamiento.

### 4.3 NOC (Number of Children)

#### Definición Detallada:

El NOC es simplemente un recuento del número de clases que heredan directamente de una clase dada (sus clases hijas inmediatas).

#### Ejemplo:



#### Propósito:

Medir la amplitud de la jerarquía de herencia en un punto específico. Es un indicador directo del potencial de **reutilización del código** de la clase padre.

#### Interpretación y Advertencias:

- **NOC alto:** Indica que la clase es un punto central de la jerarquía y se reutiliza extensivamente (se hereda de ella). Esto puede ser un buen signo de diseño modular y abstracto (ej. una clase **AbstractFactory**).
- **Riesgo de Mal Diseño:** Un NOC excesivamente alto (ej. > 10) puede señalar que la clase padre es demasiado genérica o que los principios de diseño han sido mal aplicados, creando la necesidad de forzar herencia en clases semánticamente incorrectas. El principio de Liskov (LSP) es a menudo violado en estos casos.

#### Impacto en la Calidad del Software:

- **Mantenibilidad:** Si la clase base debe ser modificada, el mantenimiento se complica. Un NOC alto implica que más clases (los hijos) deben ser re-testeados.
- **Riesgo de Propagación de Errores:** Un error introducido o descubierto en la clase padre se propaga inmediatamente a todas las clases hijas, multiplicando el impacto del defecto.

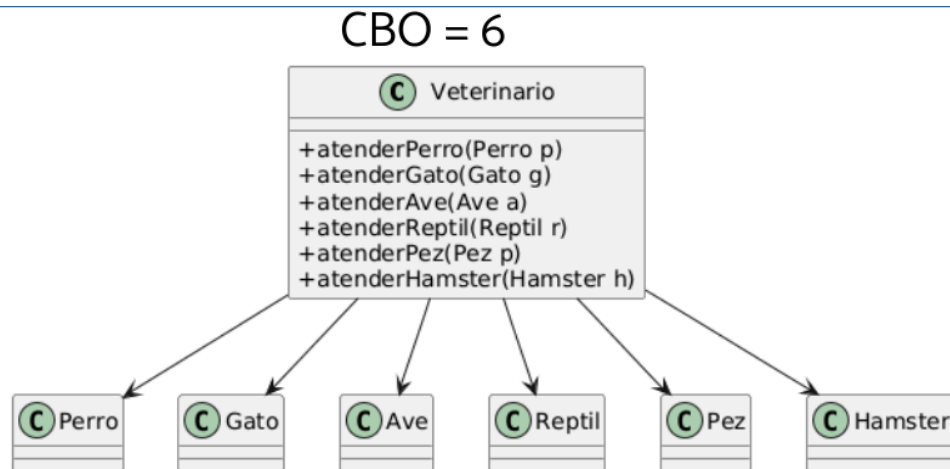
### 4.4 CBO (Coupling Between Objects)

#### Definición Detallada:

CBO es un recuento del número de clases a las que una clase dada está acoplada (depende de ellas) o que dependen de ella (la usan). El acoplamiento ocurre si una clase:

1. Utiliza atributos de la otra clase.
2. Llama a métodos de la otra clase.
3. Pasa un objeto de la otra clase como parámetro.

4. Retorna un objeto de la otra clase.
5. Declara una variable de tipo de la otra clase.

**Ejemplo:****Propósito:**

Cuantificar el nivel de interdependencia de una clase con el resto del sistema, que es un indicador clave de la modularidad.

**Interpretación y Problema Clave:**

- **CBO bajo:** Característico de un sistema modular, donde los cambios se pueden aislar dentro de módulos específicos. Es deseable para la robustez.
- **CBO alto:** Señala un sistema **rígido** o **frágil**. La clase está demasiado interconectada.

**Problema Clave: Efecto Cascada (o Efecto Dominó):**

Un CBO alto implica que un cambio en la clase actual puede requerir cambios en muchas otras clases. Inversamente, un cambio en cualquiera de las clases de las que depende la clase actual también podría forzar modificaciones en ella.

**Impacto en la Calidad del Software:**

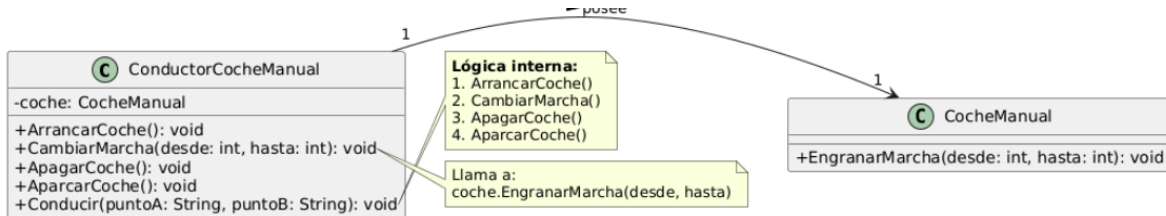
- **Modularidad:** La modularidad se reduce, haciendo que el sistema se comporte más como un monolito apretado que como un conjunto de componentes independientes.
- **Testing:** El testing unitario se vuelve difícil, ya que se requiere la simulación (mocking) de un gran número de dependencias externas.
- **Coste de Mantenimiento:** El alto acoplamiento es un motor primario del aumento del coste de mantenimiento a largo plazo.

## 4.5 RFC (Response For a Class)

### Definición Detallada:

RFC mide el tamaño del conjunto de métodos que pueden ser ejecutados en respuesta a la invocación de uno de los métodos de la clase actual.

### Ejemplo:



RS es el conjunto de todos los métodos que se pueden llamar en una clase. Como es evidente, incluye todos los métodos locales. Y todos los métodos externos que pueda llamar.

RFC = IRSI, en este ejemplo, RFC = 5 métodos internos + 1 método externo = 6

### Propósito:

Medir la complejidad del flujo de control y las interacciones potenciales de una clase con su entorno, es decir, el alcance potencial de la comunicación de la clase.

### Interpretación y Complejidad:

- **RFC bajo:** El comportamiento de la clase está autocontenido y es predecible.
- **RFC alto:** Indica que la clase dispara (o es parte de) un gran número de interacciones con el exterior. El **comportamiento es difícil de predecir** y el código de la clase se vuelve difícil de seguir.

### Ejemplo de Alto RFC:

Un método de la clase que actúa como un "coordinador" y llama secuencialmente a cinco métodos internos y tres métodos de clases externas incrementa drásticamente el RFC y, consecuentemente, su complejidad.

### Impacto en la Calidad del Software:

- **Testeabilidad:** El testing requiere no solo probar la lógica interna, sino también asegurar que todas las interacciones externas (llamadas a otros métodos) se comporten correctamente.
- **Fiabilidad:** A mayor número de interacciones, mayor es la superficie de ataque y la probabilidad de que un error en una interacción externa se propague a la clase.

## 4.6 LCOM (Lack of Cohesion of Methods)

### Definición Detallada:

LCOM es la métrica más compleja de interpretar y calcular. Busca medir el grado en que los métodos de una clase están **relacionados entre sí a través del uso de los atributos de la clase**. Una clase bien cohesionada es aquella donde cada método utiliza la mayoría de los atributos de la clase.

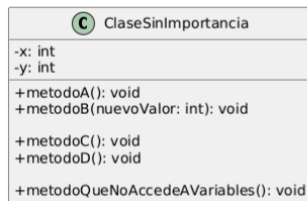
La versión original (LCOM1) se calcula como:

$LCOM = P - Q$  si  $P > Q$ , sino 0

Donde:

- **P:** Número de pares de métodos en la clase que **NO** comparten al menos un atributo de instancia.
- **Q:** Número de pares de métodos en la clase que **SÍ** comparten al menos un atributo de instancia.

### Ejemplo:



**Análisis de Cohesión (LCOM):**

- Grupo 1: {metodoA, metodoB} usan 'x'
- Grupo 2: {metodoC, metodoD} usan 'y'
- metodoD llama a metodoQueNoAccedeAVariables

Como el Grupo 1 y el Grupo 2 no comparten atributos, la cohesión es muy baja.

$LCOM = IPI - |Q|$  si  $IPI > |Q|$ , o en cualquier otro caso

$P = (a,b)$  donde a y b son métodos que **NO** comparten variables

$Q = (a,b)$  donde a y b son métodos que **COMPARTEN** variables

**0 ó 1** → Buen diseño

**2 en adelante** → Mal diseño

**Par1:** metodoA y metodoB, comparten x, por lo tanto Q++

**Par2:** metodoA y metodoC, no comparten variables, por lo tanto P++

**Par3:** metodoA y metodoD no comparten variables, por lo tanto P++

**Par4:** metodoB y metodoC no comparten variables, por lo tanto P++

**Par5:** metodoA y metodoQueNoAccedeAVariables no comparten nada, por lo que P++

**Par6:** metodoB y metodoQueNoAccedeAVariables no comparten nada, por lo que P++

**Par7:** metodoC y metodoQueNoAccedeAVariables no comparten nada, por lo que P++

### Propósito:

Medir la cohesión interna. La cohesión es la propiedad más deseable de diseño, indicando que una clase es una unidad lógica y funcionalmente enfocada.

**Interpretación y el Principio de Diseño:**

- **LCOM = 0 o 1:** Indica una clase altamente cohesionada. Idealmente, casi todos los métodos operan sobre el mismo conjunto de datos (atributos).
- **LCOM Alto:** Señala una **clase mal cohesionada**. Los métodos parecen agruparse en subconjuntos que trabajan en diferentes grupos de atributos, lo que sugiere fuertemente que la clase tiene responsabilidades múltiples y debería ser dividida en dos o más clases más pequeñas y cohesionadas (violación del SRP).

**Impacto en la Calidad del Software:**

- **Mantenibilidad y Comprensión:** Una baja cohesión obliga a los desarrolladores a buscar en una sola clase grande diferentes grupos de funcionalidad, reduciendo la mantenibilidad y haciendo la comprensión más difícil.
- **Complejidad del Sistema:** El sistema se vuelve más complejo de razonar cuando las responsabilidades se mezclan.

**4.7 Interpretación Conjunta y Perfiles de Riesgo de CK Metrics**

El valor analítico superior de las métricas CK reside en su uso **combinado** para generar perfiles de riesgo y asignar una puntuación de "salud" a las clases del sistema. Ninguna métrica individual ofrece una imagen completa; es la sinergia de sus valores la que revela problemas estructurales profundos. **Perfiles de Diseño Críticos (Ejemplos):**

| Perfil de Riesgo                   | CK Metrics Anómalas                        | Implicación en el Diseño                                                                                                   | Impacto en Calidad                                                                                                        |
|------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Monstruo de Clase</b>           | <b>WMC Alto, CBO Alto, LCOM Alto</b>       | La clase es demasiado grande, tiene demasiadas responsabilidades, y su código interno está mal organizado (baja cohesión). | Máximo riesgo de errores, mínima mantenibilidad y testeabilidad. <b>Prioridad de refactorización.</b>                     |
| <b>Clase Frágil</b>                | <b>CBO Alto, NOC Alto</b>                  | La clase depende de muchas otras y, a su vez, es heredada por muchas otras.                                                | Cualquier cambio en esta clase o en sus dependencias crea un efecto dominó masivo. Alto riesgo de propagación de errores. |
| <b>Manejador de Datos</b>          | <b>WMC Bajo, CBO Bajo, LCOM Bajo/Medio</b> | Puede ser una clase que solo contiene datos (DTO/VO) o una interfaz.                                                       | Bajo riesgo. Se espera este perfil en clases de bajo comportamiento.                                                      |
| <b>Jerarquía Profunda y Frágil</b> | <b>DIT Alto, NOC Alto</b>                  | Estructura de herencia profunda y ancha.                                                                                   | Fragilidad ante cambios en clases base. Alto coste cognitivo para entender el sistema.                                    |

Este análisis combinado permite a los equipos de desarrollo:

1. **Identificar Zonas Críticas:** Pinpoint las clases que representan el mayor riesgo de acumular deuda técnica.
2. **Priorizar Refactorizaciones:** Asignar recursos para rediseñar las clases con el perfil de riesgo más alto, optimizando el retorno de la inversión en calidad.
3. **Monitorizar la Evolución:** Utilizar las métricas a lo largo del tiempo para asegurar que el código nuevo o modificado no degrade la calidad del diseño existente.



## 5. Métricas de complejidad

Las métricas de complejidad son herramientas esenciales en la ingeniería de software que permiten ir más allá del simple conteo de líneas de código. Su principal función es **evaluar la dificultad lógica y estructural interna de un programa**. Al cuantificar esta dificultad, se obtiene una estimación directa del esfuerzo requerido para actividades cruciales del ciclo de vida del software, como:

- **Comprensión:** ¿Qué tan rápido puede un nuevo desarrollador entender la lógica?
- **Mantenimiento:** ¿Qué tan propenso es el código a romperse al introducir cambios?
- **Verificación (Testing):** ¿Cuántas pruebas son necesarias para garantizar su correcto funcionamiento?

A diferencia de las métricas de diseño (que se centran en la arquitectura, el acoplamiento y la cohesión entre módulos), las métricas de complejidad se enfocan intensamente en el **comportamiento interno y el flujo de control del código** dentro de una unidad específica (clase, método o función).

### 5.1 Complejidad Ciclomática (McCabe)

La Complejidad Ciclomática, introducida por Thomas J. McCabe Sr., es quizás la métrica de complejidad más conocida y utilizada. No es solo un número, sino una medida topológica de la estructura de control de un programa.

#### Definición Detallada:

La complejidad ciclomática  $V(G)$  mide el **número de caminos linealmente independientes** que existen en el grafo de flujo de control del programa. Un camino linealmente independiente es cualquier camino a través del programa que introduce al menos un nuevo conjunto de instrucciones de procesamiento o una nueva condición que no ha sido cubierta por los otros caminos.

#### Fórmulas de Cálculo:

Se puede calcular a partir del grafo de flujo de control  $G$ :

1.  $V(G) = E - N + 2$ 
  - $E$ : Número de aristas (conexiones o flujos) en el grafo.
  - $N$ : Número de nodos (sentencias o bloques de código) en el grafo.
2.  $V(G) = P + 1$ 
  - $P$ : Número de nodos de decisión (puntos donde el flujo se bifurca, como if, while, for, case).
3.  $V(G) = R$ 
  - $R$ : Número de regiones acotadas en el grafo (al dibujar el grafo en un plano).

#### Propósito y Utilidad:

- **Medir la complejidad lógica:** Proporciona un valor cuantitativo de la ramificación y la estructura de decisión.
- **Estimar el esfuerzo de Testing:** El valor de  $V(G)$  es el número mínimo de casos de prueba que un desarrollador debe diseñar para garantizar que **cada instrucción del programa se ejecuta al menos una vez**.
- **Identificar código de riesgo:** Módulos con alta  $V(G)$  son candidatos principales

para refactorización o reescritura.

#### Interpretación Detallada (Umbrales de Riesgo):

| Rango de V(G) | Interpretación                                | Implicación                                                                                                                                       |
|---------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 – 10        | Baja Complejidad (Zona Verde)                 | El código es fácil de entender, mantener y la cobertura de pruebas es manejable. Ideal para la mayoría de las funciones.                          |
| 11 – 20       | Complejidad Moderada (Zona Amarilla)          | El código está en un punto de equilibrio. Es funcional, pero debe monitorearse. Requiere más esfuerzo de prueba y un mantenimiento más cuidadoso. |
| > 20          | Alta Complejidad (Zona Roja - Riesgo Crítico) | Este código es altamente complejo. Es una fuerte señal de que el módulo debería ser descompuesto en unidades más pequeñas.                        |

#### Análisis Técnico Profundo:

Una complejidad ciclomática elevada no es solo una métrica abstracta; tiene consecuencias directas:

- **Gran número de decisiones condicionales anidadas:** El módulo está haciendo "demasiadas cosas" o tomando demasiadas decisiones en un solo lugar.
- **Múltiples caminos de ejecución:** El flujo de control es tortuoso, dificultando el seguimiento mental del código.
- **Mayor probabilidad de errores (Bugs):** A medida que la complejidad aumenta, la capacidad humana para predecir todas las interacciones disminuye.

El problema crítico que genera una V(G) alta es la **Imposibilidad Práctica de Testeo Completo**. Un método con  $V(G) = 50$  teóricamente requiere 50 casos de prueba unitarios solo para cubrir cada camino. Esto es un costo de testing inaceptablemente alto y, en la práctica, raramente se logra.

#### Impacto en Calidad del Software (ISO/IEC 25010):

- ↓ **Testabilidad:** El esfuerzo para verificar el código es prohibitivo.
- ↓ **Analizabilidad:** Es difícil realizar análisis de causa raíz de errores o entender el impacto de un cambio.
- ↓ **Mantenibilidad:** El riesgo de introducir efectos colaterales no deseados al modificar el código es muy alto.

### Interpretación Real y Consecuencias Económicas:

Un valor alto en esta métrica se traduce directamente en:

- **Mayor Coste de Mantenimiento Evolutivo y Correctivo:** Cada cambio o corrección de error es lento y costoso.
- **Mayor Riesgo de Introducir Bugs (Regresiones):** La naturaleza interconectada de los caminos de ejecución hace que una pequeña modificación pueda desestabilizar la funcionalidad existente.
- **Mayor Dependencia del Conocimiento del Desarrollador Original:** Solo la persona que lo escribió comprende completamente las interacciones de todos los caminos, lo que crea un "punto único de fallo" en el equipo.

### 5.2 Carga Cognitiva (Complejidad Percibida)

La Carga Cognitiva (o Complejidad Cognitiva) es una métrica más moderna que busca abordar las limitaciones de la complejidad puramente matemática. Mide el **esfuerzo mental neto requerido para entender el código**.

#### Distinción Fundamental con la Ciclomatica:

Mientras que la complejidad ciclomática se centra en los caminos de ejecución, la carga cognitiva se centra en cómo el cerebro humano procesa y retiene la información del código. Considera factores que el ojo humano y la lógica puramente matemática detectan como difíciles, tales como:

- **Profundidad de Anidamiento:** Bucles y condicionales profundamente anidados (e.g., `if (A) { if (B) { while (C) { ... } } }`) son mucho más difíciles de rastrear mentalmente que una secuencia lineal de decisiones.
- **Nombres Poco Descriptivos o Mágicos:** Uso de variables como `x`, `y`, `temp` o abreviaturas confusas.
- **Estructuras Difíciles de Seguir:** Saltos, uso excesivo de recursión compleja o patrones de diseño oscuros.
- **Abstracciones Insuficientes:** Lógica compleja contenida en un solo lugar sin ser encapsulada en funciones con nombres claros.

#### Importancia: La Perspectiva Humana:

La métrica ilustra que la complejidad no es solo matemática, sino también **humana**. Dos métodos pueden tener la misma Complejidad Ciclomatica (ejemplo: 15), pero uno puede ser estructurado y usar nombres claros, haciéndolo fácil de entender, mientras que el otro utiliza lógica oscura y anidamiento profundo, haciéndolo extremadamente confuso y propenso a errores.

La carga cognitiva cuantifica esta diferencia percibida, siendo un mejor predictor de la **velocidad de *onboarding*** de nuevos miembros y el **tiempo necesario para la revisión de código**.

#### Impacto en Calidad del Software (ISO/IEC 25010):

- ↓ **Comprensibilidad:** El esfuerzo para descifrar el propósito del código es alto.
- ↓ **Analizabilidad:** Los *code reviews* son lentos y menos efectivos.
- ↓ **Tiempo de Desarrollo/Evolución:** El tiempo invertido en entender la base de código antes de hacer un cambio eclipsa el tiempo real de codificación.

**Interpretación Real:**

Una Alta Carga Cognitiva significa:

- **Mayor Dependencia del Programador Original:** El conocimiento está tácito en la mente del autor, no explícito en el código.
- **Mayor Dificultad para Incorporar Nuevos Miembros al Equipo (Alto Coste de Onboarding):** Los nuevos desarrolladores tardan semanas en ser productivos con ese módulo.
- **Incremento del Coste de Evolución:** Cualquier nueva característica que toque el módulo complejo será retrasada o generará pánico en el equipo.

**5.3 Relación con la Deuda Técnica**

La deuda técnica es una metáfora que describe las consecuencias de priorizar la entrega rápida sobre la calidad óptima del código. Las métricas de complejidad están directamente relacionadas con la acumulación y el coste de la deuda técnica.

**El Triángulo de la Deuda Crítica:**

Cuando se combinan los siguientes tres factores, se cristaliza la deuda técnica más peligrosa, creando una situación equivalente al "**Código Heredado**" en su peor manifestación:

1. **Alta Complejidad Ciclomática:** El código es difícil de testear y tiene muchos caminos de ejecución (Alto Riesgo Lógico).
2. **Alta Carga Cognitiva:** El código es difícil de entender y mantener (Alto Coste Humano).
3. **Ausencia o Insuficiencia de Tests Unitarios:** No existe una red de seguridad que garantice que los cambios no romperán la funcionalidad.

Esta combinación genera un sistema "**Intocable y de Alto Riesgo**". El coste de modificarlo es tan alto que el equipo decide evitarlo, perpetuando el problema. Cualquier inversión futura en ese módulo estará dominada por el pago de intereses de esta deuda: más bugs, ciclos de desarrollo lentos y una moral de equipo baja. La alta complejidad, sin la mitigación de las pruebas, es el principal motor que convierte un código funcional en un pasivo financiero.

## 6. Métricas de tamaño

Las métricas de tamaño son fundamentales en la ingeniería de software, ya que ofrecen una cuantificación objetiva del volumen del producto desarrollado. Si bien son herramientas esenciales para la gestión, planificación y estimación de proyectos, su utilidad como indicadores directos de la calidad del software es limitada. Es crucial entender su aplicación, sus limitaciones y cómo deben interpretarse en el contexto global del desarrollo.

### 6.1 Líneas de código (LOC)

Las Líneas de Código (LOC) representan una de las métricas de tamaño más antiguas y directas. Se definen como el **número total de líneas que componen el código fuente ejecutable y no ejecutable** de un sistema de software. A menudo, esta métrica se desglosa en:

- **Líneas de Código Físicas:** Conteo literal de todas las líneas, incluyendo comentarios y líneas en blanco.
- **Líneas de Código Lógicas (o de Instrucción):** Conteo de sentencias o instrucciones, excluyendo comentarios y líneas en blanco, que representan el trabajo funcional real.

#### Propósito Principal

- **Medición del Tamaño del Sistema:** Proporciona una base cuantificable para dimensionar el alcance del sistema de software.
- **Estimación del Esfuerzo y Coste de Desarrollo:** Históricamente, se ha utilizado en modelos de estimación (como COCOMO) para predecir el tiempo y los recursos humanos necesarios, bajo la premisa de que un mayor número de líneas requiere más esfuerzo.

#### Limitaciones Críticas y Desventajas

El principal problema de LOC reside en su ambigüedad y su falta de correlación con atributos esenciales del software:

- **No Mide la Complejidad Intrínseca:** Un algoritmo altamente complejo puede expresarse en pocas líneas (código conciso), mientras que una tarea simple puede requerir muchas líneas en un lenguaje verboso.
- **No Mide la Calidad del Software:** La eficiencia, el rendimiento, la mantenibilidad y la robustez del código no se reflejan en el conteo de LOC. El código de alta calidad a menudo es más compacto y eficiente.
- **Alta Dependencia del Lenguaje de Programación:** Diferentes lenguajes requieren cantidades de código muy dispares para realizar la misma función.
  - *Ejemplo:* 100 líneas en un lenguaje de alto nivel y gran expresividad como Python pueden equivaler a 300 o más líneas en un lenguaje de bajo nivel como Java o C++ para lograr la misma funcionalidad.

#### Problema Crítico de Incentivos

El uso exclusivo de LOC como métrica de productividad puede generar **incentivos perversos** o malas prácticas de programación:

- **Fomento de Código Prolijo e Inflado:** Los desarrolladores pueden ser incentivados (consciente o inconscientemente) a escribir más código del necesario, aumentando artificialmente el LOC para parecer más productivos.

- **Desincentivo a la Optimización y Reutilización:** La refactorización, la reutilización de componentes o el uso de bibliotecas que reducen el LOC podrían ser vistos negativamente si el objetivo es maximizar la métrica.

### Interpretación Real y Contextual

**LOC mide la *cantidad* de artefactos de código, no la *calidad* ni el *valor funcional* entregado.**

Un sistema con un menor número de LOC que su alternativa puede ser superior porque es:

- **Más Eficiente:** Implementación algorítmica superior.
- **Más Mantenable:** Código más limpio, modular y con menor duplicación.
- **Mejor Diseñado:** Uso eficaz de abstracciones y patrones de diseño.

### 6.2 Puntos de Función (Function Points - FP) Definición Detallada

Los Puntos de Función son una métrica de tamaño del software que busca cuantificar la **funcionalidad entregada al usuario** basándose en la perspectiva del cliente, no en la del código. A diferencia de LOC, los FP son independientes de la tecnología de implementación, el lenguaje de programación o las habilidades del desarrollador.

El cálculo de FP se basa en la identificación y conteo de cinco tipos de componentes funcionales, ajustados por factores de complejidad:

- **Entradas Externas (EI):** Datos introducidos por el usuario.
- **Salidas Externas (EO):** Datos generados y presentados al usuario.
- **Consultas Externas (EQ):** Solicitudes interactivas que devuelven datos sin actualización.
- **Archivos Lógicos Internos (ILF):** Grupos de datos mantenidos dentro del sistema.
- **Archivos de Interfaz Externos (EIF):** Grupos de datos referenciados pero mantenidos por otro sistema.

### Propósito Clave

- **Estimación del Esfuerzo de Desarrollo:** Proporciona una base más sólida y temprana para estimar el esfuerzo, ya que puede calcularse desde la fase de análisis de requisitos.
- **Medición del Valor Funcional:** Cuantifica la utilidad percibida por el usuario o el valor de negocio entregado.

### Ventajas Significativas

- **Independientes del Lenguaje y la Tecnología:** Esto permite la comparación directa de la productividad entre proyectos desarrollados con tecnologías completamente diferentes.
- **Permiten Comparar Proyectos y Equipos:** Al ser una medida centrada en el negocio, facilitan la evaluación de la eficiencia de diferentes equipos o la productividad en ciclos sucesivos.
- **Útiles en Fases Tempranas:** Se pueden estimar tan pronto como se definen los requisitos.

### Limitaciones y Desafíos

- **Cálculo Complejo y Subjetivo:** Requiere la aplicación de reglas detalladas y un proceso de conteo que puede ser subjetivo si no se sigue una metodología estandarizada (como IFPUG).
- **Requiere Experiencia Específica:** El contador debe estar capacitado para identificar y categorizar correctamente los elementos funcionales.
- **Menos Útil para el Análisis Interno del Código:** Los FP solo miden la función externa; no dicen nada sobre la calidad interna del diseño, la arquitectura o la complejidad del código fuente.

### 6.3 Comparación Sintética: LOC vs. Function Points

La siguiente tabla subraya la diferencia fundamental en el enfoque de ambas métricas:

| Métrica                       | Qué Mide / Enfoque                                                        | Problema o Limitación Principal                                                           |
|-------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Líneas de Código (LOC)</b> | <b>Tamaño Técnico/Implementación</b><br>Mide el esfuerzo de codificación. | <b>No Mide Valor.</b> Es dependiente del lenguaje y puede incentivar malas prácticas.     |
| <b>Puntos de Función (FP)</b> | <b>Valor Funcional/Negocio.</b><br>Mide la utilidad entregada al usuario. | <b>Más Difícil de Calcular.</b> Requiere análisis de requisitos y experiencia específica. |

### 6.4 Interpretación en la Ingeniería de Software Moderna

Las métricas de tamaño, ya sean LOC o FP, son valiosas, pero deben ser utilizadas dentro de su contexto apropiado:

- **Estimación de Proyectos:** Son la base para los modelos de estimación de esfuerzo, coste y duración.
- **Planificación de Recursos:** Ayudan a determinar la dotación de personal y la asignación de tiempo.
- **Análisis de Productividad:** Los FP son excelentes para medir la productividad del equipo (FP por persona-mes).

**Regla de Oro:** Estas métricas cuantifican el volumen y la escala. **No deben utilizarse, en aislamiento, para evaluar la calidad interna, la mantenibilidad o la robustez del código.** La calidad es una propiedad multifacética que requiere la combinación de métricas estructurales y de proceso.

## 6.5 Relación con el Valor de Negocio y la Calidad

En el ámbito de negocio, las métricas de tamaño son esenciales para la toma de decisiones financieras y operativas:

- **Estimación de Costes:** Permiten establecer presupuestos basados en la escala del producto.
- **Planificación de Recursos:** Facilitan la gestión de la capacidad y la asignación de talentos.
- **Evaluación de la Productividad Global:** Ayudan a entender la eficiencia con la que un equipo convierte los requisitos en funcionalidades.

**Advertencia Crítica:** A pesar de su utilidad gerencial, las métricas de tamaño **no permiten evaluar directamente la mantenibilidad, la adaptabilidad o la calidad arquitectónica del diseño**. Un proyecto grande (alto LOC o FP) con mal diseño es un riesgo operativo significativo.

Por consiguiente, para una evaluación de calidad integral y efectiva, las métricas de tamaño **deben combinarse imperativamente** con métricas que midan la estructura interna y la calidad del código, tales como:

- **Métricas de Complejidad Ciclomática:** Para evaluar la complejidad de las decisiones lógicas.
- **Métricas de Diseño Orientado a Objetos (CK Metrics - Chidamber and Kemerer):** Para evaluar la modularidad, acoplamiento y cohesión.

**Métricas de Cobertura de Pruebas:** Para evaluar la robustez y el aseguramiento de calidad.



## 7. Uso práctico de métricas en proyectos reales

En el ámbito profesional de la ingeniería de software, las métricas de calidad han trascendido su papel puramente académico para convertirse en un pilar fundamental de los procesos de desarrollo modernos. No se manejan como cálculos aislados y manuales, sino como elementos integrados y automatizados dentro de un ecosistema de herramientas de análisis estático y dinámico.

El propósito principal de esta integración es la **detección temprana y proactiva de riesgos de calidad**. Al identificar patrones de código problemáticos (como alta complejidad, bajo acoplamiento o baja cohesión) justo en el momento de su introducción, los equipos pueden mitigar el riesgo de defectos, reducir el coste de mantenimiento futuro y fundamentar las decisiones técnicas con datos objetivos. Herramientas líderes en la industria como **SonarQube**, **PMD** o **Checkstyle** son las encargadas de esta monitorización continua, analizando el código en cada *commit* o integración.

### 7.1 Integración en el ciclo de desarrollo (CI/CD)

La verdadera potencia de las métricas de calidad se manifiesta a través de su integración en los *pipelines* de **Integración Continua / Despliegue Continuo (CI/CD)**. Este enfoque garantiza que la evaluación de la calidad no sea una actividad *post-mortem*, sino un control de entrada continuo.

#### Beneficios Clave de la Integración en CI/CD:

- **Detección Inmediata de Regresiones Estructurales:** Permite identificar al instante cualquier incremento en la complejidad ciclomática o el acoplamiento provocado por un cambio reciente.
- **Prevención de Deuda Técnica:** Al establecer barreras de calidad automatizadas, se evita la acumulación progresiva de código mal diseñado o de baja mantenibilidad. El sistema actúa como un "guardián" del diseño.
- **Cumplimiento Forzado de Estándares (Quality Gates):** Es posible configurar el *pipeline* para que **bloquee** la integración o el despliegue de código que no cumpla con los umbrales de calidad definidos (conocidos como *Quality Gates*).
- **Tests de Calidad Estructural:** Este proceso se asemeja a la ejecución de "tests de calidad estructural", que operan a nivel de diseño y código, complementando la cobertura de los tradicionales *tests* funcionales y unitarios.

### 7.2 Identificación y Gestión de Zonas Críticas (Hotspots)

Uno de los usos estratégicos más valiosos de las métricas es la identificación precisa de las **zonas críticas del sistema**, comúnmente denominadas *hotspots*. Estas áreas son focos de alto riesgo y representan la mayor parte de la carga de mantenimiento del proyecto.

**Características de un *Hotspot* Típico:**

| Característica           | Métrica Indicadora                       | Impacto en el Código                                                                                                       |
|--------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Alta Complejidad</b>  | Complejidad Ciclomática (CC) muy alta.   | Dificultad extrema para entender la lógica interna y las rutas de ejecución.                                               |
| <b>Alto Acoplamiento</b> | CBO (Coupling Between Objects) alto.     | El componente depende de muchos otros, haciendo que cualquier cambio interno tenga un alto riesgo de efectos colaterales.  |
| <b>Baja Cohesión</b>     | LCOM (Lack of Cohesion in Methods) alto. | La clase o módulo realiza demasiadas tareas dispares, dificultando la reutilización y el aislamiento de responsabilidades. |

**Implicaciones de los *Hotspots*:**

Estas zonas se convierten rápidamente en "código intocable" o **código heredado de facto**, incluso en sistemas nuevos. La elevada concentración de complejidad y acoplamiento se traduce en:

- **Riesgo Elevado de Errores:** La dificultad para comprender y modificar el código multiplica la probabilidad de introducir nuevos *bugs*.
- **Alto Coste de Mantenimiento:** La depuración, la extensión de funcionalidad o la refactorización consumen una cantidad desproporcionada de tiempo y esfuerzo.
- **Impedimento para la Evolución:** La resistencia al cambio en estas áreas ralentiza la capacidad del equipo para responder a nuevos requisitos o innovaciones tecnológicas.

**7.3 Umbrales (Thresholds) y el Criterio de Calidad del Código**

Para que el análisis de métricas sea accionable, es imperativo establecer **umbrales** de calidad claros. Un número por sí solo no es útil; su significado se deriva de la comparación con un valor límite que separa el código aceptable del código problemático.

**Ejemplo Detallado de Umbrales Habituales:**

| <b>Métrica</b>                            | <b>Definición Breve</b>                                                                  | <b>Umbral Recomendado (Verde)</b> | <b>Zona Crítica (Rojo)</b> | <b>Implicación del Exceso</b>                   |
|-------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------|----------------------------|-------------------------------------------------|
| <b>Complejidad Ciclomática (CC)</b>       | Número de rutas de ejecución linealmente independientes.                                 | $\leq 10$                         | $> 20$                     | Incomprensibilidad, dificultad de testing.      |
| <b>WMC (Weighted Methods per Class)</b>   | Suma de la complejidad de los métodos en una clase.                                      | $\leq 20$                         | $> 40$                     | Clase demasiado grande y compleja.              |
| <b>CBO (Coupling Between Objects)</b>     | Número de clases a las que está acoplada una clase.                                      | $\leq 5$                          | $> 10$                     | Dependencia excesiva, fragilidad al cambio.     |
| <b>RFC (Response for a Class)</b>         | Número de métodos que pueden ser invocados en respuesta a un mensaje enviado a la clase. | $\leq 50$                         | $> 100$                    | Potencialmente alta complejidad de interacción. |
| <b>LCOM (Lack of Cohesion in Methods)</b> | Grado en que los métodos de una clase usan sus atributos. (Valores bajos son mejores).   | $0 - 1$                           | $> 2$                      | Baja cohesión, la clase hace demasiadas cosas.  |

Estos valores son guías de la industria y no reglas absolutas. Su valor reside en su capacidad para establecer **criterios objetivos** que permiten a los equipos tomar decisiones consistentes sobre la salud del código, evitando la subjetividad.

**7.4 Uso Estratégico para Priorización Técnica (Refactorización)**

Las métricas proporcionan un marco basado en el riesgo para priorizar las actividades de mejora y refactorización, asegurando que el esfuerzo de ingeniería se dirija donde más valor aporta.

**Estrategia de Priorización Basada en Métricas:**

1. **Código con Valores Normales:** Requiere solo un **mantenimiento estándar** y rutinario. No se invierte esfuerzo adicional.
2. **Código Cercano a Umbrales:** Requiere **vigilancia**. Estos componentes están en riesgo de deteriorarse; se recomienda un seguimiento activo y, posiblemente, pequeñas refactorizaciones preventivas.
3. **Código Crítico (Hotspots):** Requiere **refactorización prioritaria** e inmediata. Estos son los puntos de mayor riesgo y coste.

Esta priorización evita el error común de invertir recursos en optimizar o refactorizar zonas del código que ya son saludables o que tienen un bajo impacto en el riesgo general. El foco se centra en centrar los recursos limitados en los **puntos más problemáticos y costosos** del sistema.

### 7.5 Cuantificación de la Deuda Técnica

La monitorización constante de las métricas es el mecanismo más efectivo para **cuantificar, visualizar y gestionar la deuda técnica** acumulada en un proyecto. La deuda técnica es la consecuencia de tomar atajos o de no mantener la calidad del diseño.

**Relación Métrica-Deuda Técnica:**

| Fenómeno                         | Métrica Indicadora                                                      | Significado en Deuda Técnica                                                            | Decisión Técnica Recomendada                                                               |
|----------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Deterioro del Diseño</b>      | Aumento progresivo de la Complejidad Ciclomática a lo largo del tiempo. | Se están introduciendo soluciones cada vez más enrevesadas y menos claras.              | <b>Refactorizar componentes específicos</b> para simplificar la lógica.                    |
| <b>Pérdida de Modularidad</b>    | Incremento constante del CBO.                                           | El sistema se está volviendo monolítico, con componentes hiper-dependientes.            | <b>Rediseñar módulos</b> para mejorar el aislamiento y la separación de responsabilidades. |
| <b>Escalamiento de Problemas</b> | Acumulación de múltiples métricas críticas en un solo <i>hotspot</i> .  | La falta de atención a un área crítica está comprometiendo la viabilidad a largo plazo. | <b>Inversión estratégica</b> para reescribir o aislar la funcionalidad.                    |

Al medir la deuda técnica de esta manera, los equipos pueden presentar un caso de negocio sólido a la dirección para justificar el tiempo necesario para la refactorización, transformando una decisión técnica en una **decisión de gestión de riesgo financiero** y operativo.

## 8. Métricas y negocio: impacto económico (ROI)

Uno de los pilares fundamentales de la ingeniería de software moderna es la comprensión profunda de cómo la **calidad técnica** de un sistema impacta directamente en la **salud económica y la viabilidad a largo plazo** de una organización. Las métricas de software no son meros indicadores técnicos; son el lenguaje que permite a los ingenieros traducir propiedades intrínsecas del código en **costes tangibles, riesgos de negocio y oportunidades de inversión**. Esta sección profundiza en la conexión ineludible entre la excelencia técnica y el Retorno de la Inversión (ROI).

### 8.1 Relación Intrínseca entre Calidad, Complejidad y Coste de Mantenimiento

Existe una relación causal y documentada entre las características internas del diseño y la estructura del código y los costes operativos futuros. Los atributos clave del diseño impactan directamente en la mantenibilidad y, por ende, en el coste total de propiedad (TCO) del software:

- **Complejidad Ciclomática (McCabe) y Coste de Entendimiento:** Una alta complejidad en las funciones y módulos (más caminos de ejecución) se traduce en un **mayor tiempo y esfuerzo** para que un desarrollador (incluso el autor original) pueda comprender, modificar o depurar el código. Esto ralentiza la incorporación de nuevas funcionalidades y la corrección de errores.
- **Acoplamiento (CBO - Coupling Between Objects) y Riesgo de Propagación:** Un alto acoplamiento indica que un cambio en un módulo tiene una alta probabilidad de requerir cambios en múltiples módulos dependientes. Esto aumenta el **riesgo de introducir nuevos defectos** y multiplica el tiempo necesario para realizar una modificación "aislada".
- **Cohesión (LCOM - Lack of Cohesion in Methods) y Dificultad de Reutilización:** Una baja cohesión significa que un componente o clase realiza múltiples tareas no relacionadas. Esto dificulta la reutilización del código, complica la prueba unitaria y hace que el módulo sea frágil ante cambios que solo afectan a una de sus responsabilidades.

Un sistema que exhibe de forma crónica una alta complejidad, un acoplamiento excesivo y una baja cohesión, inevitablemente presenta:

- **Mayor densidad de defectos:** Los errores se ocultan más fácilmente en código enrevesado.
- **Mayor tiempo de *Time-to-Market* (TTM):** El desarrollo de nuevas funcionalidades es lento y tortuoso.
- **Aumento exponencial del coste de evolución y *bug fixing*:** Cada cambio es más caro y arriesgado.

Este círculo vicioso es la manifestación directa de la **Deuda Técnica**. La Deuda Técnica como Coste Acumulado

La **Deuda Técnica** surge de la elección consciente o inconsciente de atajos de diseño o implementaciones rápidas pero subóptimas. Estas decisiones, que ofrecen un beneficio a corto plazo (lanzamiento más rápido), generan un **interés acumulativo** en forma de costes futuros. Las métricas actúan como el balance que cuantifica esta deuda, señalando los módulos donde el interés (coste de mantenimiento) es prohibitivamente alto.

## 8.2 La Dominancia del Coste de Mantenimiento

Estudios de la industria consistentemente demuestran que el mantenimiento (adaptativo, correctivo, perfectivo y preventivo) constituye la partida de gasto más significativa en el ciclo de vida del software, a menudo oscilando **entre el 60% y el 80% del coste total del proyecto**.

Las métricas de calidad son el principal predictor de este coste:

| Métrica Crítica                | Impacto en el Coste                                                         | Implicación Económica                                                                 |
|--------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Alta Complejidad</b>        | Más tiempo para el análisis y la comprensión del código.                    | <b>Incremento en horas/persona</b> para <i>bug fixing</i> y desarrollo.               |
| <b>Alto Acoplamiento (CBO)</b> | Mayor necesidad de pruebas de regresión y riesgo de efectos colaterales.    | <b>Mayor coste de <i>testing</i></b> y aumento en incidentes de producción.           |
| <b>Baja Cohesión (LCOM)</b>    | Dificultad para aislar y reutilizar funcionalidades; necesidad de rediseño. | <b>Coste de refactorización estructural</b> o, peor, la necesidad de una reescritura. |

## 8.3 El ROI Positivo de la Inversión en Calidad del Software

El uso estratégico de métricas permite transformar el gasto en calidad en una **inversión justificada** con un Retorno de Inversión (ROI) medible.

**El ciclo virtuoso del ROI en Calidad:**

1. **Inversión:** Utilizar métricas para identificar los módulos con mayor deuda técnica (p. ej., alta complejidad, alto acoplamiento) y asignar tiempo de ingeniería a la **refactorización** o el **rediseño**.
2. **Reducción de Costes Operativos:** Tras la refactorización, el código es más legible, modular y fácil de probar.
  - **Menor TTM:** El tiempo para implementar nuevas *features* se reduce drásticamente.
  - **Menos Defectos:** La calidad inherente del código disminuye la tasa de errores en producción.
3. **Resultado - ROI Positivo:**
  - **Aumento de la Productividad:** El equipo dedica menos tiempo a pelearse con código heredado y más tiempo a generar valor nuevo.
  - **Disminución de Costes a Largo Plazo:** Se reduce el coste de mantenimiento y el riesgo de fallos catastróficos.
  - **Mejora de la Moral y Retención:** Los desarrolladores prefieren trabajar en código limpio, mejorando la satisfacción y retención del talento.

## 8.4 Gestión del Riesgo y la Toma de Decisiones Estratégicas

Las métricas ofrecen una evaluación cuantitativa del **riesgo técnico**, que es vital para la toma de decisiones a nivel gerencial, no solo técnico.

El ejemplo clásico es la modificación de un módulo "crítico":

- **Escenario de Alto Riesgo:** Módulo con **Complejidad Ciclomática > 50** y **Alto Acoplamiento a Componentes Críticos del Negocio**.
- **Análisis del Riesgo:** Cualquier cambio en este módulo tiene una alta probabilidad de introducir regresiones, requerirá un esfuerzo de *testing* desproporcionado y podría paralizar una parte clave del negocio.
- **Influencia en Decisiones Estratégicas:**
  - **Gestión de Proyectos:** Posponer una funcionalidad que dependa de este módulo hasta que se haya mitigado el riesgo.
  - **Asignación de Recursos:** Priorizar un *sprint* de refactorización preventiva antes que el desarrollo de nuevas *features*.
  - **Arquitectura:** Determinar si el módulo debe ser desacoplado o, en casos extremos, si se requiere una reescritura de esa porción del sistema.

## 8.5 Impacto de la Calidad en la Sostenibilidad y Competitividad del Negocio

Desde una perspectiva puramente empresarial, la baja calidad del software representa una amenaza directa a la sostenibilidad operativa y la competitividad en el mercado:

- **Lentitud en la Evolución:** Un sistema con mala calidad frena la capacidad de la empresa para responder a las demandas del mercado o a los cambios regulatorios.
- **Recursos Ineficientes:** Más ingenieros son necesarios solo para mantener el *status quo* (caza de *bugs*), desviando recursos valiosos de la innovación.
- **Riesgo de Fallo de Negocio:** La baja calidad aumenta la probabilidad de incidentes graves de producción, lo que puede resultar en tiempo de inactividad, pérdida de clientes y daños a la reputación.

Las consecuencias empresariales pueden ser devastadoras: **pérdida de cuota de mercado, incremento de costes operativos fijos y la eventual necesidad, costosa y riesgosa, de una reescritura completa del sistema** (que suele ser el resultado final de la deuda técnica descontrolada).

## 8.6 Métricas como Herramienta de Gobernanza y Alineación Estratégica

Las métricas de calidad son el puente de comunicación entre la tecnología y el negocio, permitiendo que las decisiones técnicas se alineen directamente con los objetivos estratégicos de la organización (gobernanza).

- **Definición de Estándares:** Establecer umbrales de calidad mínimos que todo el código debe cumplir antes de ser integrado (p. ej., "ningún módulo debe superar una complejidad ciclomática de 15").
- **Criterios de Aceptación:** Incluir métricas de calidad como parte de los criterios de aceptación de una *feature* o un proyecto. Un proyecto no se considera "terminado" si introduce una deuda técnica inaceptable.
- **Monitorización Proactiva:** Usar paneles de control basados en métricas para monitorizar la salud del sistema a lo largo del tiempo, identificando tendencias negativas (crecimiento de complejidad, aumento de acoplamiento) antes de que se conviertan en crisis.

Al integrar las métricas en la gobernanza, la calidad del software se convierte en un **activo estratégico** gestionado, no un resultado accidental.



## 9. Gobernanza y Dashboard de Calidad

La calidad del software trasciende la mera implementación técnica de buenas prácticas de programación. Su máximo impacto y valor se materializan al integrarse plenamente en un **sistema de gobernanza** robusto y bien definido. En este contexto estratégico, las métricas dejan de ser indicadores técnicos aislados y se transforman en elementos críticos de control, seguimiento y, fundamentalmente, de **toma de decisiones** a nivel ejecutivo y organizativo.

### 9.1 Gobernanza de la Calidad del Software: Fundamentos y Alcance

La gobernanza de la calidad del software (GCS) es el marco que asegura que la producción y evolución del software estén alineadas con los objetivos de negocio, las normativas y la estrategia corporativa. Establecer una GCS implica definir y aplicar rigurosamente tres pilares esenciales:

- **Estándares de Calidad:** Definición clara y documentada de los criterios mínimos aceptables para la arquitectura, el diseño, el código, la documentación y el rendimiento del software. Esto incluye la adopción de modelos reconocidos (como ISO/IEC 25000 o modelos internos).
- **Políticas de Desarrollo:** Reglas operativas que dictan cómo se debe construir, probar, desplegar y mantener el software. Estas políticas deben ser prescriptivas y obligatorias para todos los equipos.
- **Mecanismos de Control y Aseguramiento:** Procesos y herramientas automatizadas (y manuales) para auditar, verificar y validar continuamente el cumplimiento de los estándares y políticas.

El objetivo primario de la GCS es garantizar una evolución del software que sea **sostenible, predecible y conforme** con las expectativas del negocio. En esta estructura, la **importancia de las métricas** es insustituible, ya que permiten:

- **Definir Criterios Objetivos de Calidad:** Pasar de declaraciones vagas ("el código debe ser bueno") a umbrales medibles ("la complejidad ciclomática no debe exceder 15").
- **Evaluar el Cumplimiento de Estándares:** Proporcionar evidencia cuantificable sobre la adhesión a las políticas de la organización.
- **Detectar Desviaciones Oportunas:** Identificar rápidamente áreas donde la calidad se está deteriorando antes de que generen un impacto crítico.

Sin la base de datos que proporcionan las métricas, la gobernanza degenera en percepciones subjetivas, debates basados en opiniones y una gestión de riesgos reactiva. Con métricas, la gestión de la calidad se convierte en una disciplina proactiva y basada en **evidencia cuantificable**.

### 9.2 Dashboards de Calidad: La Visualización del Estado del Sistema

Los **dashboards de calidad** son la interfaz visual y operativa de la gobernanza. Son herramientas de monitorización y agregación de datos diseñadas para proporcionar una visión en tiempo real del estado de salud del proyecto o del portfolio de sistemas. Su valor reside en la capacidad de transformar grandes volúmenes de datos brutos en información accionable.

Herramientas líderes en el mercado, como SonarQube, ofrecen paneles que consolidan métricas clave de diversos aspectos de la calidad del código, incluyendo:

- **Complejidad:** Medidas como la complejidad ciclomática, indicando la dificultad de entender y modificar el código.
- **Cobertura de Tests:** El porcentaje de código fuente que es ejercitado por las pruebas automatizadas (unitarias, integración). Un indicador directo de la robustez y capacidad de refactorización.
- **Duplicación de Código (Code Duplication):** Identificación de bloques de código idénticos o muy similares, una fuente primaria de errores y un obstáculo para el mantenimiento eficiente.
- **Deuda Técnica (Technical Debt):** Una estimación del costo implícito de las deficiencias de calidad detectadas (errores de diseño, *bugs* latentes, malas prácticas).

Estos cuadros de mando son vitales porque permiten a los diferentes *stakeholders* (desarrolladores, arquitectos, *project managers*, ejecutivos) realizar tareas críticas:

- **Identificar Tendencias:** Observar si la calidad está mejorando o deteriorándose a lo largo de las iteraciones de desarrollo.
- **Detectar Áreas Problemáticas (Hotspots):** Focalizar los esfuerzos de refactorización y revisión en los módulos más riesgosos o de menor calidad.
- **Tomar Decisiones Informadas:** Justificar inversiones en refactorización, tiempo adicional para pruebas o la necesidad de formación específica para los equipos.

### 9.3 Quality Gates: La Defensa Activa contra la Deuda Técnica

Un elemento táctico y preventivo crucial dentro de la gobernanza son los **Quality Gates** (Puertas de Calidad). Estos no son solo herramientas de medición, sino mecanismos de control que imponen un **punto de control obligatorio** en el flujo de integración continua/entrega continua (CI/CD). Un *Quality Gate* establece un conjunto de **umbrales mínimos de calidad** que el código debe satisfacer **antes** de ser fusionado en la rama principal (*mainline*), desplegado a un entorno superior o liberado.

Ejemplos típicos de condiciones para un *Quality Gate*:

- **Límite de Complejidad:** La complejidad ciclomática de cualquier nuevo componente o función debe estar por debajo de un valor límite predefinido (e.g., 10 o 15).
- **Cobertura de Tests Mínima:** El código nuevo o modificado debe estar cubierto por pruebas en un porcentaje superior a un umbral (e.g., 80% o 90%).
- **Tolerancia Cero a Fallos Críticos:** Ausencia total de vulnerabilidades de seguridad, *bugs* o *code smells* catalogados como críticos o de alta prioridad.
- **Mantenibilidad:** La calificación general de mantenibilidad debe mantenerse o mejorar en comparación con la versión anterior.

#### La Importancia Estratégica de los Quality Gates:

- **Previenen la Introducción de Deuda Técnica:** Al actuar como un filtro, impiden que el código deficiente contamine el repositorio principal, atacando la deuda técnica desde su origen.
- **Aseguran un Nivel Mínimo de Calidad Consistente:** Garantizan que el nivel de calidad del *software* nunca caiga por debajo del estándar organizacional, independientemente del equipo o desarrollador.

- **Automatizan el Control del Código:** Mueven la responsabilidad del control de calidad de la revisión manual y subjetiva a un proceso automatizado y objetivo.

Este enfoque transforma la calidad de un objetivo deseable a un **requisito obligatorio** del proceso de desarrollo.

#### 9.4 Métricas como Herramienta de Control Continuo

La gobernanza moderna se basa en el principio de **Control Continuo**. En un ciclo de desarrollo ágil y DevOps, donde las liberaciones son frecuentes y los cambios constantes, la evaluación de la calidad debe ser igualmente continua.

Este enfoque de "evaluación en cada *commit*" permite:

- **Detectar Problemas en Tiempo Real:** Las métricas se actualizan instantáneamente, permitiendo al desarrollador corregir el defecto segundos después de haberlo introducido.
- **Evitar la Acumulación de Errores:** Al solucionar los problemas tan pronto como aparecen, se previene la peligrosa acumulación de defectos que luego requerirán costosas y largas refactorizaciones.
- **Mantener la Calidad a lo Largo del Tiempo:** La calidad se gestiona como un proceso continuo y no como una fase terminal del proyecto, asegurando que el producto se mantenga estable a medida que evoluciona.

#### 9.5 Relación Crítica con Sistemas Reales y Legado

La carencia o debilidad en la gobernanza de la calidad tiene consecuencias directas y graves en el mundo real, especialmente en entornos empresariales con sistemas grandes y complejos:

- **Sistemas "Intocables":** Cuando no hay métricas, ni tests, ni estándares claros, los desarrolladores temen modificar el código, por miedo a introducir fallos catastróficos. El sistema se convierte en una "caja negra" con un alto riesgo operativo.
- **Deuda Técnica Descontrolada:** La falta de *Quality Gates* y el desinterés por las métricas permiten que la deuda técnica crezca exponencialmente, haciendo que el costo de mantenimiento y la velocidad de desarrollo se vuelvan insostenibles.
- **Incumplimiento de Estándares Regulatorios:** En industrias reguladas (finanzas, salud), la falta de gobernanza y métricas claras puede llevar al incumplimiento de normativas de seguridad, auditoría o protección de datos, resultando en sanciones severas.

La ausencia de control y medición transforma el sistema en una "caja negra" inmanejable, replicando los peores problemas del **código heredado** de manera continua, dificultando su evolución y aumentando dramáticamente el riesgo operativo. Una GCS efectiva, basada en métricas y visualizada en *dashboards*, es la única vía para garantizar la longevidad, la mantenibilidad y el éxito estratégico del *software*.

## 10. Bibliografía

- [ISO/IEC 25010. \(2011\). \*System and software quality models\*. ISO.](#)
- [Chidamber, S. R., & Kemerer, C. F. \(1994\). \*A Metrics Suite for Object-Oriented Design\*. IEEE Transactions on Software Engineering.](#)
- [Fenton, N. E., & Bieman, J. \(2014\). \*Software Metrics: A Rigorous and Practical Approach\*. CRC Press.](#)
- [McCabe, T. J. \(1976\). \*A Complexity Measure\*. IEEE Transactions on Software Engineering.](#)